

API Calling to ChatGPT Lab Report

Automated Product Listing Generator

How the API integration works

The project connects a Python script to the OpenAI API using an API key stored securely in a .env file. Product metadata is loaded from products_metadata.csv, while product images are read from the local product_images folder. Each image is converted into a base64 string so it can be sent inside the API request together with a structured prompt. The prompt asks the vision-capable model to analyze the image and metadata, then return a JSON product listing with a title, description, key features, and SEO keywords. The final script loops through multiple products, parses each API response, and saves the generated listings into generated_product_listings.json.

Challenges faced

The main challenges were related to the Python environment and package installation. The active environment initially did not contain required packages such as python-dotenv and openai, which caused ModuleNotFoundError messages. This was resolved by installing packages with python -m pip install to ensure they were added to the same environment running the script. Another challenge was to make the prompt for GPT strict enough to make the model return JSON instead of just an explanation.

Quality of generated listings

The generated listings were clear, structured, and suitable for an e-commerce platform. The model successfully used both image details and metadata to create product-specific copy. For example, it generated titles and descriptions for items such as a navy blue shirt, blue jeans, and a silver women's watch. The output included persuasive descriptions, practical feature lists, and relevant SEO keywords. Overall, the quality was stronger than a simple template-based approach because the listings reflected visible product details such as color, style, season, and use case.

Potential improvements

Future versions could process larger batches, add retry logic for temporary API failures, and validate the JSON schema more strictly before saving results. The script could also include cost tracking, progress logs, and configurable batch sizes from the command line. Listing quality could be improved by adding different brand tones, such as premium, casual, technical, or budget-focused. Another useful improvement would be storing results in a database or exporting directly to an e-commerce platform instead of only saving a JSON file.